

DEGREE PROJECT IN ELECTRICAL ENGINEERING,
SECOND CYCLE, 30 CREDITS
STOCKHOLM, SWEDEN 2021

Predicting Buffer Status Report (BSR) for 6G Scheduling using Machine Learning Models

KTH Thesis Report

QI ZHANG

KTH ROYAL INSTITUTE OF TECHNOLOGY
SCHOOL OF ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

Authors

Qi Zhang <qizhan@kth.se>
Information and Network Engineering
KTH Royal Institute of Technology

Examiner

György Dán <gyuri@kth.se>
KTH Royal Institute of Technology

Supervisor

Sladana Josilo <josilo@kth.se>
KTH Royal Institute of Technology

Marios Daoutis <marios.daoutis@ericsson.com>
Ericsson AB

Alexandros Nikou <alexandros.nikou@ericsson.com>
Ericsson AB

Abstract

In 6G communication, many state-of-the-art machine learning algorithms are going to be implemented to enhance the performances, including the latency property. In this thesis, we apply Buffer Status Report(BSR) prediction to the uplink scheduling process. The BSR does not include information for data arriving after the transmission of this BSR. Therefore, the base station does not allocate resources for the new arrival data, which increases the latency. To solve this problem, we decide to make BSR predictions at the base station side and allocate more resources than BSRs indicate. It is hard to make an accurate prediction since there are so many features influence the BSRs. Another challenge in this task is that the time intervals are tremendously short (in the order of milliseconds). In other traffic predictions, the traffic data in a long term, such as in a week and month, can be used to predict the periodicity and trend. In addition, many external features, such as the weather, can boost the prediction results. However, when the time is short, it is hard to leverage these features.

The datasets provided by Ericsson are collected from real networks. After cleaning the data, we convert the time series forecasting problem into a supervised learning problem. State-of-the-art algorithms such as Random Forest(RF), XGboost, and Long Short Term Memory(LSTM) are leveraged to predict the data arrival rate, and one K-Fold Cross-Validation is followed to validate the models. The results show that even the time intervals are small, the data arrival rate can be predicted and the downlink data, downlink quality indicator and rank indicator can boost the forecasting performance.

Keywords

Buffer Status Report, 6G scheduling, Machine Learning, Uplink Traffic Prediction, Wireless Communication

Sammanfattning

I 6G-kommunikation kommer många toppmoderna maskinin lärnings algoritmer att implementeras för att förbättra prestanda, inklusive latensegenskapen. I den här avhandlingen vill vi tillämpa Buffer Status Report (BSR) förutsägelse för schemaläggningsprocessen för upplänkning. BSR innehåller inte information för data som anländer efter överföring av denna BSR. Därför tilldelar basstationen inte resurser för den nya ankomstdatan, vilket ökar latensen. För att lösa detta problem bestämmer vi oss för att göra BSR-förutsägelser på basstationssidan och tilldela fler resurser än vad BSR anger. Det är svårt att göra en exakt förutsägelse eftersom det finns så många funktioner som påverkar BSR. En annan utmaning i denna uppgift är att tidsintervallen är oerhört korta (millisekunder). I andra trafikprognoser kan trafikdata på lång sikt, som under en vecka och månad, användas för att förutsäga periodicitet och trend, och många externa funktioner, såsom väder, kan öka förutsägelseresultaten. Men när tiden är kort är det svårt att utnyttja dessa funktioner.

Dataset som tillhandahålls av Ericsson samlas in från riktiga nätverk. Efter rengöring av data konverterar vi tidsserieprognosproblemet till ett övervakat inlärningsproblem. Toppmoderna algoritmer som Random Forest (RF), XGboost och LSTM (Long Short Term Memory) utnyttjas för att förutsäga data ankomst astigheten och en K-Fold Cross-Validation följs för att validera modellerna. Resultaten visar att även tidsintervallen är små, datainkomsthastigheten kan förutsägas och nedlänksdata, kvalitetsindikator för nedlänk och rangindikator kan öka prognosprestandan.

Nyckelord

Buffert status rapport, 6G-schemaläggning, Machine Learning, Uplink Traffic Prediction, Wireless Communication

Acknowledgments

I would like to thank my supervisors Sladana Josilo, Marios Daoutis and Alexandros Nikou who give me patient guidance. Also I want to thank my examiner Gyorgy Dán for his kind help.

A thanks to my friend Wang ShuChan who helped me during my trip to Stockholm during the pandemic.

Contents

1 Introduction **1**
1.1 Background . . . 1
1.2 Problem . . . 1
1.3 Purpose and Goals . . . 2
1.4 Contribution . . . 2
1.5 Research Methodology . . . 2
1.6 Ethics and Sustainability . . . 3
1.7 Thesis Structure . . . 3

2 Background **5**
2.1 BSR . . . 5
2.1.1 Short BSR . . . 7
2.1.2 Long BSR . . . 7
2.1.3 Regular BSR . . . 7
2.1.4 Padding BSR . . . 8
2.1.5 Periodic BSR . . . 8
2.2 RNN and LSTM Models . . . 8
2.2.1 RNN . . . 9
2.2.2 LSTM . . . 9
2.3 Ensemble Learning . . . 11
2.3.1 Bagging . . . 11
2.3.2 Boosting . . . 12
2.4 Regression Tree . . . 12
2.5 Random Forest . . . 14
2.6 XGboost . . . 14
2.6.1 Regularized Learning Objective . . . 14
2.6.2 Gradient Tree Boosting . . . 15

3 Problem Formulation and Related Work **19**
3.1 Problem Formulation 19
3.2 Related Work 20

4 Implementation and Results **23**
4.1 Data Collection 23
4.2 Data Preprocessing 24
4.2.1 K-Fold Cross-Validation 27
4.2.2 R^2 scores 28
4.3 Random Forest 29
4.4 XGboost 33
4.5 LSTM 34

5 Discussion **37**

6 Conclusions and Future Work **39**
6.1 Conclusions 39
6.2 Limitations 40
6.3 Future work 41

References **43**

List of Figures

2.1 Uplink scheduling process 6
2.2 Short BSR MAC control element 7
2.3 Long BSR MAC control element 7
2.4 An example of the RNN unit 9
2.5 The information transmission in RNN units 10
2.6 The structure of LSTM unit 10
2.7 The prediction result from one 2-tree ensemble model. 15
2.8 An example of the tree and the corresponding structure score. . 17

3.1 Periodic BSR 20
3.2 Padding BSR 20

4.1 Distribution of the length of time series for dataset C. The X-axis shows the range of length of data series, and the Y-axis shows the corresponding frequency. 26
4.2 Distribution of BSR intervals for the active UEs in dataset C. The X-axis shows the range of intervals between two adjacent BSRs, and the unit is ms.The Y-axis shows the corresponding frequency. 27
4.3 Relative error performance as a function of the number of trees for dataset C. The X-axis shows the number of trees in the RF system.The Y-axis shows the relative error, which is the ratio of current error and maximum error. 29
4.4 Relative error performance as a function of the number of features in a tree for dataset C. The X-axis shows the number of features in one tree.The Y-axis shows the relative error, which is the ratio of current error and maximum error. 30
4.5 Prediction results of RF for respective datasets. The X-axis shows the index number of test data. The Y-axis shows BSR rates, expressed in bit/ms. 31

4.6 Performance as a function of the number of trees in XGboost for dataset C 32
4.7 Prediction results of XGboost for respective datasets. The X-axis shows the index number of test data. The Y-axis shows BSR rates, expressed in bit/ms. 34
4.8 Prediction results of LSTM for respective datasets. The X-axis shows the index number of test data. The Y-axis shows BSR rates, expressed in bit/ms. 35

List of Tables

4.1 Features in dataset A . . . 23
4.2 Features in dataset B . . . 24
4.3 Structure of the input and output data . . . 26
4.4 Parameters in RF . . . 29
4.5 Error of RF for dataset A . . . 31
4.6 Error of RF for dataset B . . . 31
4.7 Error of RF for dataset C . . . 31
4.8 Errors of XGboost for dataset A . . . 33
4.9 Errors of XGboost for dataset B . . . 33
4.10 Errors of XGboost for dataset C . . . 33
4.11 Errors of LSTM for dataset A . . . 36
4.12 Errors of LSTM for dataset B . . . 36
4.13 Errors of LSTM for dataset C . . . 36

5.1 Performance of different algorithms . . . 37
5.2 R^2 scores for dataset C . . . 38

List of acronyms and abbreviations

BSR : Buffer Status Report
ARIMA : AutoregressiveIntegrated Moving Average
UE : User Equipment
RF : Random Forest
LSTM : Long Short-Term Memory
URLLC : Ultra-Reliable Low Latency Communications
SR : Schedule Reques
gNb : next Generation Node
PUCCh : Physical Uplink ControlChanne
UL-SCH : Uplink Shared ChanneL
PDCCH : Physical Downlink Control Channel
LCG : Logical Channel Group
GARCH : eneralized Auto Regressive Conditional Heteroscedasticity
FF-NN : Feed-Forward Neural Networks
AR : Auto Regressive
RNN : Recurrent Neural Networ
NLP : Natural Language Processing
CART : classification and regression tree
SFN : system frame number
RNTI : Radio Network Temporary Identifier
DQI : DL Quality Indicator
RI : Rank Indicator
MAE : Mean Absolute Error
RMSE : Root Mean Squared Error

Chapter 1

Introduction

1.1 Background

With the continuous development of communication, a growing number of devices have access to the Internet, producing massive traffic data. In addition, in the future, the traffic scenarios such as uplink videos, various applications, and web pages will contribute a significant portion of the traffic data. Thus the research on uplink scheduling is of considerable interest. The contemporary uplink scheduling mechanism is based on the BSR, which may increase the latency for some data and can be improved by machine learning algorithms. Furthermore, in 6G, state-of-the-art machine learning algorithms will be implemented to enhance the uplink performance metrics such as the latency performance.

1.2 Problem

One of the shortcomings of BSR is that it does not include information about newly arrived data that comes during the interval between the time when BSR is sent and when the uplink data is transmitted. As a result, the data arriving during this interval can not be forwarded immediately and has to wait for the subsequent round uplink transmission, which increases the latency. The prediction of BSR can solve this problem. The questions in this thesis are:

- Which models/algorithms could be implemented to predict the BSR?
- Which features can be leveraged in the prediction in the case of short intervals between the BSRs?

- How can one preprocess the datasets to enhance the performance?

1.3 Purpose and Goals

If the base station can predict the data arrival rate and allocate more resources than the BSR indicates, the newly arrived data can be sent with previous data, which may reduce the latency. Compared to the actual volume of the newly arrived data, if the predicted volume is less than that, there is still some data having to wait for the subsequent round transmission. If the expected volume is greater than the actual volume, the additional resource is wasted. Therefore, our purpose is to make a precise prediction of the volume of the newly arrived data and decrease the end-to-end latency.

1.4 Contribution

In this thesis, we implement state-of-the-art machine learning algorithms to predict the BSR rate. To our knowledge, we are the first group to make BSR predictions in such short time intervals. The results show the possibility of making precise BSR predictions. In addition, it indicates that features such as downlink data, quality indicator, and rank indicator can enhance the prediction performance.

1.5 Research Methodology

There are two datasets provided by Ericsson, which are collected from real network traffic in different companies. A K-Fold Cross-Validation is leveraged to test the machine learning model. Typically, for the time series prediction, the Autoregressive Integrated Moving Average(ARIMA) model is widely implemented. However, one of the challenges in the case of predicting the BSRs is that there are many User Equipments(UEs), and each UE has different properties. Furthermore, the BSRs occur in bursts. As a result, ARIMA is not suitable for the prediction, and thus we transfer the time series forecasting into a supervised learning problem. Another challenge in this task is that the time intervals are tremendously short (in the order of milliseconds). In other traffic predictions with long time intervals, such as the workday and weekend, many internal features can be used to predict can predict the periodicity and trend, and many external features, such as the weather, can boost the prediction

results. However, when the time is short as in the case of BSRs, it is hard to leverage these features.

The time when the uplink data is uploaded is unknown in the datasets. Therefore, instead of predicting the volume of arrival data, we predict the data arrival rate. We divide the data generated from one UE into several parts for the reason that the relationships of two subsequent BSRs are weak when the time intervals are long. Finally, we decide to apply state-of-the-art algorithms such as Random Forest(RF), XGboost and Long Short-Term Memory(LSTM) to solve this problem. As we will see later, these algorithms achieve good performance.

1.6 Ethics and Sustainability

As we mentioned before, the traffic data is growing at high speed, and in the new generation of communication, people expect that the latency in communication can be dramatically reduced. According to Ericsson's research, the uplink data will be a large part of the total traffic data. Therefore, our enhancement in the uplink scheduling is beneficial to the improvement of the user experience. In addition, our prediction is on the base station side and no data from higher layers such as the application layer is leveraged in our models. Hence, there is little privacy risk for the users. Since the lower latency can improve the efficiency of communication,our work may boost the building of resilient infrastructure and inclusive and sustainable industrialization.

1.7 Thesis Structure

This thesis is divided into several chapters. In Chapter 2, background information about the buffer status prediction process in the uplink scheduling mechanisms is introduced. Chapter 3 presents how the chosen machine learning algorithms work. Chapter 4 focuses on data processing and presents some of the main results obtained for three different models of BSR rate prediction. These models are further discussed and compared with each other in Chapter 5. Finally, Chapter 6 presents the conclusions and future work.

Chapter 2

Background

This chapter provides basic background information about the buffer status prediction process in the uplink scheduling mechanisms.

With the continuous development of communication, a growing number of devices have access to the Internet, producing massive traffic data. Ericsson (2021) [1] forecasts that the global mobile data traffic is growing by a factor of 4.5, and the total monthly data volume will reach 226EB in 2026. Additionally, in the future, the traffic scenarios such as uplink videos, various applications, and web pages will contribute a significant portion of the traffic data. Therefore, the research on uplink scheduling is of considerable interest.

The Ultra-Reliable Low Latency Communications (URLLC) is one of the new generation communication requirements. In order to achieve URLLC, especially when the traffic data is massive, many domains can be optimized. In this thesis, BSR in the uplink scheduling process is analyzed, and the predictions of BSR are used to improve the latency performance.

2.1 BSR

In what follows we present the basics related to uplink scheduling and BSRs. Unless otherwise noted, the main concepts and analytical expressions are the same as those in [2]. If some data is arriving at the UE side, the uplink scheduling process is as follows. First, the UE sends the Schedule Request(SR) message to the gNB(next Generation NodeB) over the Physical Uplink Control Channel (PUCCH). The UE informs gNB via the SR whether the uplink resource is needed for the Uplink Shared Channel(UL-SCH) transmission, but it does not tell gNB the volume of data that needs to be sent (this is reported via the BSR).

After the gNB receives the SR message, it sends the UL grant (Downlink Control Information) over Physical Downlink Control Channel(PDCCH) to the UE side. Since the gNB does not know the volume of data on the UE side, the UL grant is usually small, and UE can only send the BSR information.

Subsequently, the UE needs to inform gNB about the volume of data in its uplink buffer that is waiting to be sent. Based on the received BSR, the gNB can decide how much uplink resources to allocate to the UE. Once the grant from gNB is received, the data in the UE's buffer can be sent. Figure 2.1 illustrates described the uplink scheduling process.

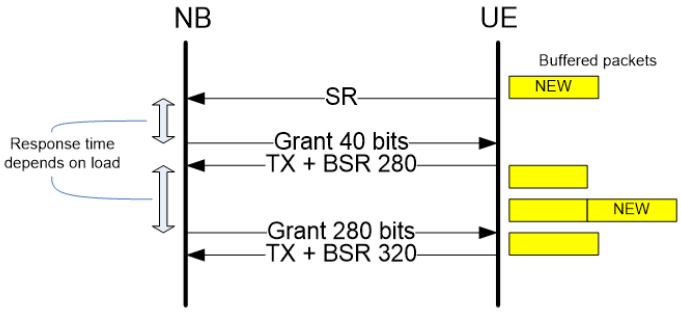


Figure 2.1 – Uplink scheduling process

Depending on the service, UE may establish many radio bearers (one logical channel per bearer). If one BSR is reported for each logical channel, there will be considerable resource overhead. In order to avoid this, the concept of Logical Channel Group (LCG) is proposed, which is a group of logical channels with similar scheduling requirements. The BSR is reported for each LCG instead of each channel, which can prevent overhead problems. The study of BSR can inform us the BSR trigger information, which will benefit the prediction process.

The BSR is reported via the BSR MAC Control Element in the MAC layer. Based on the length, there are two formats of BSRs, the Short BSR and Long BSR. Depending on the trigger mode, the BSRs can be divided into three types, the regular BSR, padding BSR, and periodic BSR, which will be described in the following.

2.1.1 Short BSR

A short BSR contains only two fields, LCG ID and buffer size, as shown in Figure 2.2. The length of LCG ID is 2 bits and the length of buffer size is 6 bits, which indicates all the data in the LCG.

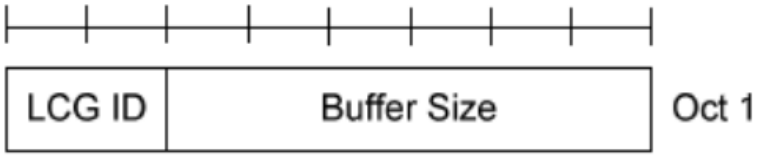


Figure 2.2 – Short BSR MAC control element

2.1.2 Long BSR

A long BSR contains information for all four LCGs, which is longer than Short BSR. Different from short BSR, there is no LCG ID, since in long BSR, the position is used to present the LCG ID, which is shown in 2.3. For example, the first 6 bits present the buffer information for the first LCG.

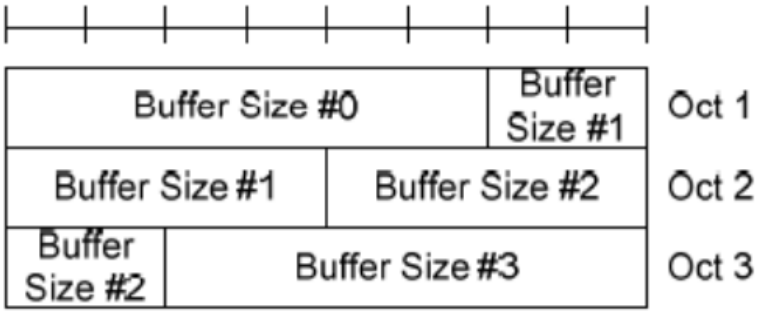


Figure 2.3 – Long BSR MAC control element

2.1.3 Regular BSR

One can identify three types of regular BSR. When the UE's uplink buffer is empty, there is no uplink data that can be sent over any logical channels in the LCG. Hence, at this point, the UE will trigger a BSR if any logical channel in the LCG has new data coming in.

Given the priority of the logical channels, there is another type of BSR. During the time when the UE has sent a BSR and is waiting for UL Grant. If there is some new data arriving at the logical channel whose priority is higher than any logical channel in the LCG, a BSR is triggered immediately.

In order to increase the robustness of the BSR, there is a retransmission mechanism for BSR. There is a timer named retxBSR-Timer. If the UE has sent the BSR and does not receive any UL Grant, it will resend this BSR after the retxBSR-Timer expires. The UE reboots the retxBSR-Timer every time it sends a BSR.

2.1.4 Padding BSR

When the UE has been granted more resources than it needs to send the data, the extra resource will be filled by the padding bits. However, if the additional resource is sufficient enough, then it will be used to send a BSR, which we call padding BSR. Whether the BSR is short or long depends on the volume of the extra resource.

2.1.5 Periodic BSR

The UE periodically updates its buffer state to the NB, and it does so through periodic BSR. Similar to the retxBSR-Timer, there is a periodicBSR-Timer. The UE will send a BSR if the timer expires. For example, if there is a large file to be sent, there is new data arriving at the buffer when the UE is waiting for the UL Grant. Hence, the UE needs to constantly update the volume of uplink data that needs to be transmitted.

2.2 RNN and LSTM Models

The LSTM model is widely used in supervised learning, which is based on the RNN. LSTM is first proposed in [3]. Compared to RNN, LSTM has better performance in long sequence processing. For long sequence training, the RNN suffers from gradient vanishing and gradient explosion problems. On the contrary, the LSTM can avoid this Long-Term Dependencies problem. Each LSTM unit has three control gates to choose information to forget, forward to the next unit, and serve as the output. Unless otherwise noted, the main concepts and analytical expressions are the same as those in [4].

2.2.1 RNN

The Recurrent Neural Network (RNN) is a Neural Network used to process sequence data, which is based on the research in [5]. Different from the traditional neural network, the RNN does not have the problem to make predictions based on previous information. For example, different from the traditional neural network, RNN can be used to successfully predict the next words based on previous sentences in Natural Language Processing (NLP). However, the weakness of RNN is the lack of ability to connect information from a long time ago, which is called the long-term dependency problem.

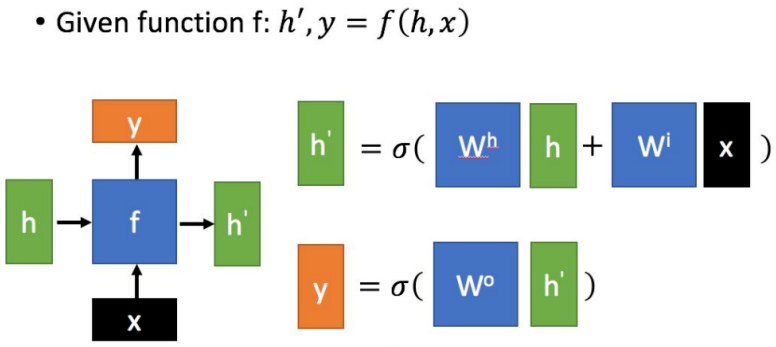


Figure 2.4 – An example of the RNN unit

The unit of RNN is shown in Figure 2.4, where $\sigma()$ is the sigmoid function. $\mathbf{x}$, $\mathbf{y}$ represent the input and output of the data in the current state, respectively. $\mathbf{h}$ represents the input from the last node, and $\mathbf{h}'$ represents the output to the next node. $\mathbf{W}$ and $\mathbf{b}$ are the corresponding weight matrix and the bias, respectively. The structure of RNN unit is simpler than that of LSTM, as we will see later. There is only one state information $\mathbf{h}$ to transfer, and the process is shown in Figure 2.5.

2.2.2 LSTM

In order to solve the problems of long-term dependencies, the LSTM model is proposed. Compared with normal RNN, LSTM can perform better for long sequences.

The unit of LSTM is different from the unit of RNN. In RNN, each unit only has to transfer the state information $\mathbf{h}$ to the next unit. The LSTM unit has to transfer the cell state $\mathbf{c}^t$ and hidden state $\mathbf{h}^t$ to the next unit. The cell state $\mathbf{c}^t$ changes slowly and hidden state $\mathbf{h}^t$ changes fast. The structure of LSTM unit

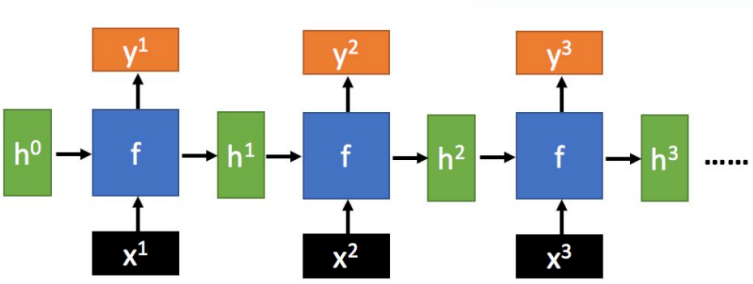


Figure 2.5 – The information transmission in RNN units

is shown in Figure 2.6 . The $\odot()$ represents Hadamard Product, which is the multiplication of the corresponding elements in the matrix. In what follows, we will describe the internal structure of the LSTM unit.

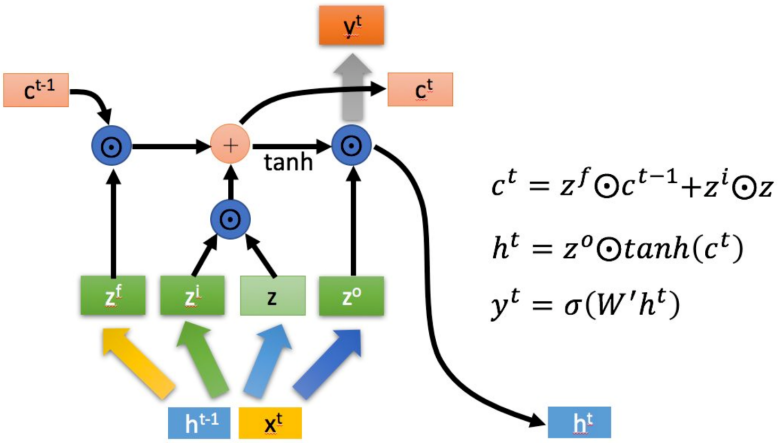


Figure 2.6 – The structure of LSTM unit

The unit selectively forgets some cell state information from the previous unit, which is achieved by z^f, where

$$z^f = \sigma(W_f[h^{t-1}, x^t] + b_f)). \tag{2.1}$$

Despite the forgetting process, the unit has to select some information from the input x^t to remember. This process is controlled by z^i, where

$$z^i = \sigma(W_i[h^{t-1}, x^t] + b_i)). \tag{2.2}$$

The output of the current state should also be selected, which is achieved

through z^o, where

$$z^o = \sigma(W_o[h^{t-1}, x^t] + b_o)). \tag{2.3}$$

All z^f, z^i, z^o serve as gate control status, and thus $sigmoid$ functions are selected. z serves as the input of the system such that the $tanh$ activation function is used

$$z = tanh(W[h^{t-1}, x^t] + b). \tag{2.4}$$

Finally, we can express the outputs of the LSTM unit as

$$c^t = z^f \odot c^{t-1} + z^i \odot z. \tag{2.5}$$

$$h^t = z^o \odot tanh(c^t). \tag{2.6}$$

$$y^t = \sigma(W' h^t). \tag{2.7}$$

Gating states are used to control the transmission states, which can forget unimportant information and remember useful information. Compared to RNN, whose structure is more straightforward, LSTM has better performance for tasks that require long-term memory.

2.3 Ensemble Learning

Ensemble learning models are widely used in machine learning. Compared to other machine learning models, ensemble learning is trying to train multiple models for the training data and combine them under specific policies. Typically the ensemble learning models have better performance than the base models [6, 7]. There are many popular ensemble strategies, such as bagging and boosting.

2.3.1 Bagging

Bagging is one of the most known parallel ensemble learning methods, which is firstly proposed in [8]. Bagging is directly based on the bootstrap sampling method. Given a dataset that contains many samples, we first randomly take out one sample from the sample set, and then put the sample back to the initial dataset. The selected sample is still able to be taken out next time. We take k rounds to extract k training sets, each of them containing n samples. In the process of bootstrap, the k training sets are independent of each other due to

the fallback sampling. Each training set is used to train a model, and totally k base models are obtained from k training sets.All the base models are used to predict the test set and the ensemble strategy aggregates the prediction results.

Bagging has several advantages. First, the approach is parallelizable. Second, each base model can be generated separately, independently of each other. Finally, bagging can reduce the variance. The main disadvantage of bagging is that it cannot reduce the bias significantly.

2.3.2 Boosting

Similar to bagging, boosting is also a widely used ensemble learning method. Boosting is proposed for the question posted in [9], which is "Can a set of weak learners create a single strong learner?". In a boosting model, firstly, a base model is trained from the initial training set. Then the distribution of training samples is adjusted according to the performance of the base learner. More specifically, the weights of the correctly classified samples are reduced and the misclassified samples' weights are increased to make the misclassified samples more prominent in the future. Then, the next basic learner is built based on the samples with the adjusted distribution. This process is repeated until the number of base learners reaches the pre-specified value T, and finally, the T base learners are weighted and combined.

Compared with bagging, where the training set is selected in the original set, and the training sets selected from the original set are independent of each other, the training set in boost is unchanged, and only the weights are altered according to the previous results. The base models in boosting are closely correlated, and thus they cannot be trained in parallel. Therefore, the boost models typically can reduce the bias.

2.4 Regression Tree

Decision trees are widely implemented in many fields, such as data mining and machine learning. This is mainly because the decision trees can be built in an easy and fast way. Based on data types, decision trees are divided into two categories: classification decision trees and regression decision trees.

The decision tree is composed of nodes and diredcted edges. There are two types of nodes:internal nodes and leaf nodes. An internal node represents one feature, and a leaf node represents one value or category. For the classification or regression task, one feature is task at the root node, and the samples are assigned to its child nodes according to the test results. Each child node

corresponds to a value of the feature. The samples are recursively tested and allocated until the leaf node is reached. The decision tree can be seen as a method that can be used for dividing the space into many parts, and in this context, each leaf node represents a unique region in the space. More specifically, based on the features of the sample, each leaf node is allocated to one region during the decision process.

The Classification And Regression Tree(CART) is one of the decision tree methods, firstly proposed in [10], and widely used in the classification and regression tasks. In what follows we introduce the regression trees, and the method of how to find the splitting point and determine the output values for leaf nodes.

We denote by $D = \{x_i, y_i\}$ a dataset with n samples with m features, i.e., $|D| = n, x_i \in R^m, y_i \in R$. One regression tree divides the space into K regions, which we denote by $R_1, R_2, ...R_K$, respectively. We use c_i to denote the output value for region R_i. Using this notation, we can express the regression output of this model as

$$f(x) = \sum_{i=1}^{n} c_i I(x \in R_i), \tag{2.8}$$

where I is an indicator function that is equal to 1 if $x \in R$ is true and 0 otherwise. Once the regions are determined, we use square error to define the loss. For a fixed region R_m, the loss can be then expressed as $\sum_{x_i \in R_m}(y_i - f(x_i)^2)$. In order to minimize the loss function, the optimal output should be the average value of samples in the region, and can be expressed as $\hat{c}_m = avg(y_i|x_i \in R_m)$.

As for the splitting process, we notate the j-th feature of the input as the splitting variable $x^{(j)}$, and its value as the splitting point s. The space is then divided into two regions: $R_1(j,s) = \{x_i|x_i^{(j)} \leq s\}$ and $R_2(j,s) = \{x_i|x_i^{(j)} > s\}$. We find the optimal splitting variable $x^{(j)}$ and splitting point s by solving

$$min_{j,s}[min_{c_1} \sum_{x_i \in R_1(j,s)} (y_i - c_1)^2 + min_{c_2} \sum_{x_i \in R_2(j,s)} (y_i - c_2)^2], \tag{2.9}$$

where c_1 and c_2 are the output values for $R_1(j,s)$ and $R_2(j,s)$.

2.5 Random Forest

Random Forest(RF) is one of the ensemble learning models. In the following we discuss the main characteristics of RF. Unless otherwise noted, the main concepts and analytical expressions are the same as those in [11].

The base models in RF are all CARTs, and there are n CARTs that are combined in the RF system. For the n CARTs, n different training sets are generated from the original dataset via the bootstrap sampling method, which is the same as bagging. The randomness in RF is not only in the selection of the training data, but also in the training process. For each sample, there are m input features. However, for the training of each node of the CARTs, only p input features are randomly selected, where $p < m$. Finally, we get n base models, and the result of the classification is the one chosen by the majority of trees. The result of the regression is the average value.

2.6 XGboost

XGboost is a recently proposed tree boosting machine learning algorithm that shows good prediction and regression performances. It is a popular state-of-the-art machine learning algorithm and is widely used in real life. For example, in the machine learning competition website Kaggle, there are 29 winning solutions (all of them rank top 3 for the given competition), and XGboost is used in more than half of the solutions. Unless otherwise noted, the main concepts and analytical expressions are the same as those in [12].

2.6.1 Regularized Learning Objective

In what follows we will introduce the details concerning the XGboost. First, in a tree ensemble model, there are K regression trees. The regression tree allocates a continuous value w_i to each leaf i. For the m-dimension input $\mathbf{x}$, the related index of leaf in this tree is notated as $q(\mathbf{x})$, and the corresponding value is $w_{q(\mathbf{x})}$.

We denote by $D = \{\mathbf{x}_i, y_i\}$ a dataset with n samples with m features, i.e., $|D| = n, \mathbf{x}_i \in \mathbb{R}^m, y_i \in \mathbb{R}$. . The output for the k-th regression tree is $f_k(\mathbf{x}_i) = w_{q(\mathbf{x})}$. For the input $\mathbf{x}_i$, the prediction from this tree ensemble model is given by

$$\hat{y}_i = \phi(\mathbf{x}_i) = \sum_{k=1}^{K} f_k(\mathbf{x}_i). \tag{2.10}$$

Figure 3.4 shows the outputs from a 2-tree ensemble model.

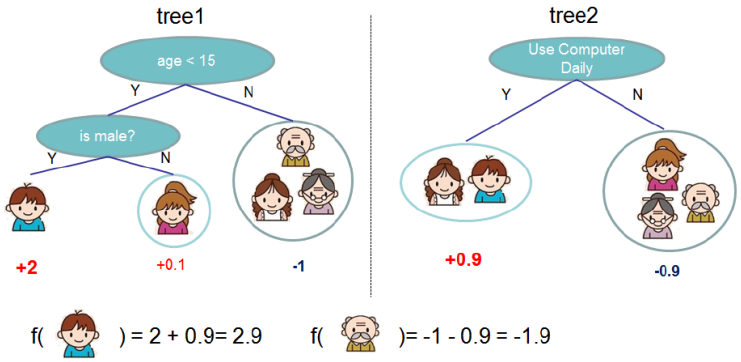


Figure 2.7 – The prediction result from one 2-tree ensemble model.

For the training process, there is a loss function l, which is differentiable and convex. The loss function is used to quantize the difference between the target y_i and the prediction result $\hat{y}_i$. When evaluating the performance of a solution, it is important to take into consideration not only the loss function but also the complexity of the model. The complexity Ω of the system in the case of XGboost is defined as

$$\Omega(f) = \gamma T + \lambda ||\omega||^2, \tag{2.11}$$

where T is the number of leaves in the tree, γ and λ are the regularization parameters.

The learning process is to minimize the objective function defined as

$$\mathcal{L}(\phi) = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k). \tag{2.12}$$

2.6.2 Gradient Tree Boosting

During the training process, we can define the predicted value as the sum of predicted values from k trees. At the t-th iteration, the prediction value can be expressed as

$$\hat{y}_i = \sum_{k=1}^{t} f_k(\mathbf{x}_i) = \sum_{k=1}^{t-1} f_k(\mathbf{x}_i) + f_t(\mathbf{x}_i). \tag{2.13}$$

Hence, we can rewrite the objective function as:

$$\mathcal{L}(\phi)^t = \sum_{i=1}^{n} l(y_i, \hat{y}_i^{t-1} + f_t(\mathbf{x}_i)) + \Omega(f_t). \tag{2.14}$$

During the optimizing process, second-order approximation can be applied to (3.14)

$$\mathcal{L}(\phi)^t \approx \sum_{i=1}^{n} [l(y_i, \hat{y}_i^{t-1}) + g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i)] + \Omega(f_t), \tag{2.15}$$

where $g_i = \partial_{\hat{y}^{t-1}} l(y_i, \hat{y}_i^{t-1})$ and $h_i = \partial^2_{\hat{y}^{t-1}} l(y_i, \hat{y}_i^{t-1})$.

In order to simply this term, the constant terms are removed

$$\tilde{\mathcal{L}}(\phi)^t = \sum_{i=1}^{n} [g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i)] + \Omega(f_t). \tag{2.16}$$

Next, we define the sample set of leaves j as $I_j = \{i|q(x_i) = j\}$, and further rewrite the objective function as

$$\tilde{\mathcal{L}}(\phi)^t = \sum_{i=1}^{n} [g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i)] + \gamma T + \frac{1}{2} \sum_{i=1}^{T} \omega_i^2. \tag{2.17}$$

$$\tilde{\mathcal{L}}(\phi)^t = \sum_{j=1}^{T} [(\sum_{i \in I_j} g_i)\omega_i) + \frac{1}{2} (\sum_{i \in I_j} h_i + \lambda)\omega_i^2)] + \gamma T. \tag{2.18}$$

If the tree structure is fixed, the optimal weight for leaf j is given by

$$\omega^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}, \tag{2.19}$$

and the optimal loss function value can be expressed as

$$\tilde{\mathcal{L}}(\phi)^t = -\frac{1}{2} \sum_{j=1}^{T} \frac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T. \tag{2.20}$$

Equation (3.19) can be used to value the structure of trees, which means that the structure with lower values has better performance. Fig 4.6 shows the structure score of the given tree.

Equation (3.19) can also serve as the guideline for the leaf split. For a leaf,

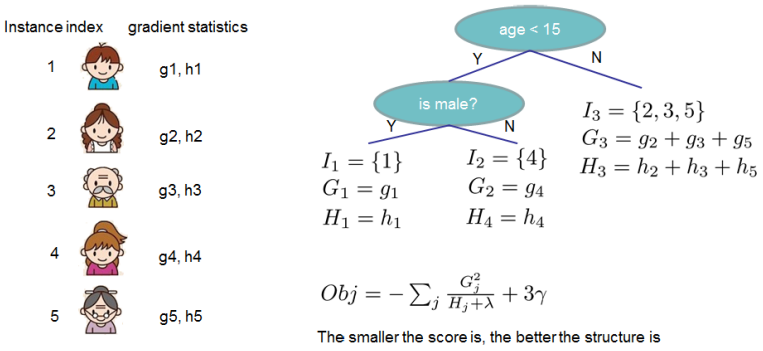


Figure 2.8 – An example of the tree and the corresponding structure score.

I is the original sample set, and I_L and I_R are the sample sets after the split. Hence, the loss reduction via this split can be expressed as

$$\mathcal{L}_{split} = \frac{1}{2}[\frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} + \frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda}] - \gamma. \quad (2.21)$$

This function can be used to find the optimal split and reduce loss function.

Chapter 3

Problem Formulation and Related Work

3.1 Problem Formulation

In what follows we analyze the latency performance for the uplink data. Figure 2.4 and 2.5 summarize the latency in each uplink process.

In the beginning, the buffer of UE is empty. New data is arriving at the UE side and getting packetized. Then the SR message is sent by UE, and it is delivered to the NB after the transmission time. The NB will send the UL grant information back if it finishes the processing of SR information. This grant resource is small, therefore, the UE can only send BSR information back. Once the NB receives the BSR, it allocates as much resource as specified by the UE in the BSR and sends a grant message to inform the UE that the resources have been allocated. The UE will transmit as much data as the NB allocates later.

If there is still some data in the UE side waiting to be transmitted after the first uplink data transmission, it has to wait for another BSR-UL grant-data transmission round. However, the BSR cannot be triggered too frequently since too frequent BSR transmissions would lead to a high channel overhead. We assume there is no BSR packet loss and the priority of newly arrived data is not higher than the previous data. Hence, only padding BSR and periodic BSR can be triggered.

K2 time illustrated in Figures 2.4 and 2.5 is the time between UL grant reception in DL and UL data transmission. We use t_K and t_T to denote the K2 time and the transmission delay between the UE and NB, respectively. For the periodic BSR, the BSR is triggered when the periodicBSR-Timer expires. We

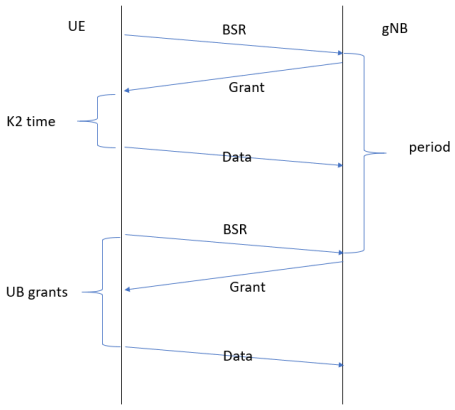


Figure 3.1 – Periodic BSR

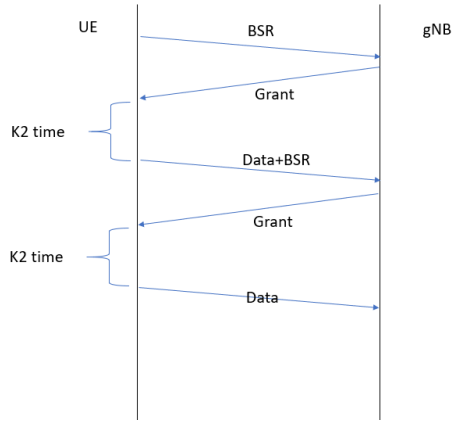


Figure 3.2 – Padding BSR

denote the time interval between the two BSRs as t_P. For the padding BSR, since the BSR and data are sent together, the time interval between the two BSRs is $t_K + 2t_T$.

It is important to note that there is a $t_K + 2t_T$ long time interval between the time instances when the UE sends BSR and the data gets transmitted. Unfortunately, the BSR does not include information about the data that arrives during this time interval, and the newly arrived data has to wait for the next round.

To decrease the latency, one of the possible solutions is that based on the previous history data, the NB predicts how much data will arrive during the time interval between the BSR and UL data transmission. Based on this prediction, the NB can allocate more resources than the BSR indicates. Consequently, the data that arrives after the BSR is sent can be transmitted without waiting for the next round. Therefore, the uplink latency can be dramatically reduced if the data volume can be accurately predicted, which was the original goal of this thesis.

3.2 Related Work

There are several works that focus on the uplink scheduling process[13, 14]. The authors in [13] investigate the impact of the uplink scheduling time on the end-to-end latency.The simulation results show that the latency may decrease as K2 time increases. On the one hand, the increase of K2 allows for the transmission of larger portions of the newly arrived data. On the other hand, the previous data has to wait for a longer time due to the increase

of K2. Therefore, when choosing the value for K2 time, one needs to take into consideration the trade-off between the latency of data arriving before and after the UL grant reception. Different from the authors in [13] who consider the scheduling time, the authors in [14] implement the BSR prediction in the uplink scheduling. This paper focuses on the satellite mobile communication system, where the transmission delay is high. The BSR prediction is performed at the user side, and the volume BSR indicates is larger than the data in the UE's buffer. Consequently, the data arrives during the BSR transmission, and UL data transmission can be sent together with the previous data, which reduces the end-to-end latency. The shortcoming in this paper is a simple prediction model, which assumes that the traffic data follows an already known distribution. Furthermore, the authors only focus on the satellite mobile communication. However, for other traffic scenarios such as video streaming and web browing, the predictions for BSR are more difficult since the intervals between the BSRs are shorter.

There are many approaches to make the predictions, and one of the most common algorithms is ARIMA[15, 16]. The authors in [15] use ARIMA for network traffic prediction.The ARIMA models are used to predict the inner connection between the time series. For each training set, the coefficients of ARIMA are calculated. A regression tree is built to analyze the relationship between coefficients and features such as the data collection time and the cell IDs. Finally, the coefficients get re-estimated and applied into the ARIMA prediction process. Time intervals for predictions considered in [15] are, however, much larger than the time intervals considered in this thesis. Similarly, the authors in [16] use ARIMA and Generalized AutoRegressive Conditional Heteroskedasticity (GARCH) for the network traffic prediction. They show non-linear time series model such as ARIMA/GARCH outperforms linear model due to the bursty property of the traffic flow.

Except for ARIMA, Neural Networks is widely used in time series prediction [17, 18, 19]. The authors in [17] use the Feed-Forward Neural Networks (FF-NN) and Classic-Auto Regressive(AR) to make network traffic predictions. They try to bridge the statistical and deep learning-based approaches by using AR to explain the parameters in the Neural Networks. Therefore, the network structure considered in [17] is quite simple. The authors in [20] firstly propose the LSTM model, which is based on Recurrent Neural Network (RNN). It is hard for RNN to make decisions based on obsolete information, which is called the problem of long-term dependencies. To solve this problem, each LSTM unit uses three control gates to choose information to forget, forward to the next unit, and serve as the output. The authors in [18] apply LSTM to make cellular

traffic prediction and traffic model classification. Their simulation results show that LSTM outperforms ARIMA. The datasets in [18] are collected from real traffic network, and the time granularity is 10 seconds, which is much longer than the time intervals in uplink scheduling. The authors also find that the downlink data can enhance the prediction performance. The work in [19] shows that LSTM can be leveraged to predict the uplink throughput, indicating that LSTM is able to make accurate predictions even when the time intervals are small. Different from the traffic data volume prediction in our thesis, the authors in [19] focus on the prediction of the uplink throughput, and only data generated from web-based real-time streaming applications is analyzed. The authors in [21] proposes one network traffic prediction model with millisecond granularity e based on a meta-learning scheme. Meta-learning schemes are recently proposed for making reinforcement learning. The simulation results show this model can make accurate predictions. However, the datasets used in this paper only include a few traffic scenarios.

Similar to Neural Networks, ensemble learning models [22, 23] are common methods for time series forecasting. The authors in [24] firstly proposed RF, which is one of the most widely used ensemble learning models. RF is based on bagging, which can reduce the variance. The authors in [22] implement various algorithms to predict the stock index. The resluts show that RF outperforms other models in variance. XGboost, another Scalable Tree Boosting System, is widely used in the prediction areas, which is proposed in [12]. XGboost is based on boosting, which can reduce bias. The authors of [23] use XGboost to predict the short-time traffic and they show that their solution can achieve good prediction performances. However, in [23], only the transportation traffic instead of network traffic flow is analyzed.

Chapter 4

Implementation and Results

4.1 Data Collection

The datasets are provided by Ericsson, and they include the real traffic data collected from companies. There are two datasets, which we refer to as A and B, respectively.

Table 4.1 – Features in dataset A

variables	Description
CELL	ID of the CELL a UE connected with
TS	ID of the subframe
SFN	ID of the system frame
UE ID	the ID of each UE
HARQ	the ARQ feedback
TDS	DL Transport Data Size in bits
SR	Schedule Request
BSR	Buffer Status Report

Dataset A is formed from the data collected from a real company, and it includes the information from 10 cells and more than 1000 UEs. The subframe number of each message is recorded, and thus the time intervals of packets are known since each subframe lasts for 1 ms. The detailed description of the features in dataset A is shown in Table 4.1.

Although there are plenty of data recorded in the dataset, many of the UEs rarely upload their data to the cell. To be more specific, less than 10% of the UEs send the BSR information frequently. Another observation is that the BSRs tend to occur in bursts. In some time intervals, there are many BSRs

generated by one particular UE. However, for the same UE, there may be a long time interval during which no BSR is sent.

Table 4.2 – Features in dataset B

variables	Description
RNTI	Radio Network Temporary Identifier
DTM	DL Transmission Mode
DM	DL Modulation
DQI	DL Quality Indicator
RI	Rank Indicator
PUCCH	RX power for PUCCH signal in dBm
PUSCH	RX power for PUSCH signal in dBm

Dataset B is formed from the data collected from another company. Dataset B contains different features compared to dataset A, and unfortunately, some of the records are missing. Table 4.2 shows the detailed description of the features in dataset B.

4.2 Data Preprocessing

Data processing is a step prior to the implementation of the system model. The first challenge facing the data processing is that there are many UEs and each UE has different properties. In addition, as we mentioned before, the BSRs occur in bursts. For a UE, there may be an extended period that no BSR is sent since no uplink data is needing to be forwarded, and the UE keeps silent. As a result, there are many time series, and their lengths vary significantly. Considering the difference in the coefficients and size of each time series, the ARIMA model is unsuitable for predicting the provided datasets. Hence, we decide to convert the time series prediction problem into the supervised learning problem and implement state-of-the-art algorithms such as XGboost, Random Forest and LSTM to solve the supervised learning problems.

For dataset A, there are some essential features such as the timestamp. In 5G, each frame lasts for 10 ms, and consists of 10 subframes. Since the volume of downlink data for every subframe is recorded, it is easy to obtain the time intervals between BSRs. One of the shortcomings is that for uplink data, only the value of BSR is recorded. Consequently, the the length of time intervals between the moments when a UE sends BSR and transmits the uplink data is unknown. In addition, even intervals between BSRs for the same UE vary

significantly. Instead of predicting the volume of arrived data, we predict the data arrival rate, and we express the average data arrival rate u_t for the t-th time interval as

$$u_t = \frac{BSR_t}{NF_t - NF_{t-1}}, \tag{4.1}$$

where BSR_t is the data volume that the t-th BSR indicates and NF_t is the serial number of subframe when the t-th BSR is sent. The average data arrival rate u_t is expressed in bit/ms.

Similarly, we express the average downlink data rate d_t for the t-th time interval as

$$d_t = \frac{\sum_{i=NF_{t-1}}^{NF_t-1} TDS_i}{NF_t - NF_{t-1}}, \tag{4.2}$$

where TDS_i is the size of downlink data at the i-th subframe. The average data arrival rate d_t is expressed in bit/ms.

Dataset B contains more features than dataset A. However, many of the features in dataset B are related to the physical layer or not entirely recorded. We decide to add two features to our input, that is, the average of DL Quality Indicator and Rank Indicator since we know that the priority class of the data impacts the BSR. Therefore, we express the average of DL Quality Indicator dqi_t as

$$dqi_t = \frac{\sum_{i=NF_{t-1}}^{NF_t-1} DQI_i}{NF_t - NF_{t-1}}, \tag{4.3}$$

where the DQI_i is the DL Quality and Rank indicator for the i-th subframe. Similarly, we express the average of Rank Indicator ri_t as

$$ri_t = \frac{\sum_{i=NF_{t-1}}^{NF_t-1} RI_i}{NF_t - NF_{t-1}}, \tag{4.4}$$

where RI_i is the Rank indicator for the i-th subframe. Now, we can convert the time series forecasting problem into a supervised learning problem. First, we need to divide the data from every UE into many parts. If the interval between two BSRs is too long (here we assume more than 40ms), the relationship between these BSRs is week, and consequently, we assume they are generated from different applications.

For data generated from each application, the length distribution of the data series is shown in Fig 4.1. It is important to mention that the choice of the input value of the time window T has great importance. For a time series whose length is L, we can get $L - T$ labeled data samples. Given the

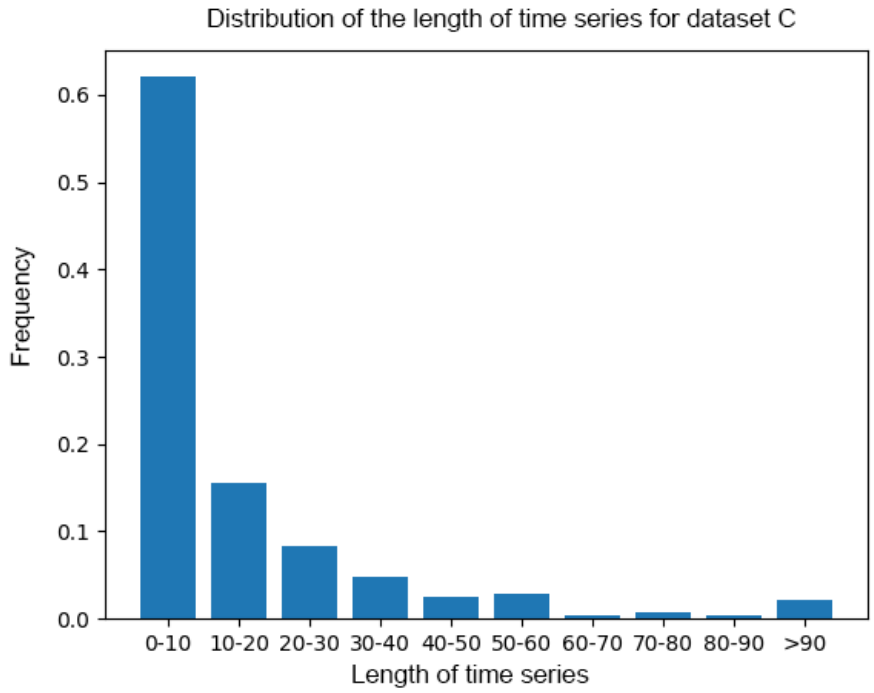


Figure 4.1 – Distribution of the length of time series for dataset C. The X-axis shows the range of length of data series, and the Y-axis shows the corresponding frequency.

Table 4.3 – Structure of the input and output data

Configuration	**Input**	**Output**
BSR	$[u_{t-T}, u_{t-T+1}, ..., u_{t-1}]$	u_t
BSR and DL	$[u_{t-T}, ., u_{t-1}, d_{t-T+1}, ., d_t]$	u_t
ALL FEATURES	$[u_{t-T}, ., u_{t-1}, d_{t-T+1}, ., d_t, dqi_t, ri_t]$	u_t

distribution of the length of time series, we decide to set the time window $T = 5$. Consequently, the most straightforward approach is to use BSR rates in the previous T intervals to predict the BSR in the next interval. In addition, the distributions of downlink and uplink data series are co-related. Hence, we can add the downlink data into the input. Also, for dataset B, the downlink quality indicator and rank indicator can present the priority class of the data, which benefits the prediction. The structure of the input is shown in Table 4.3.

After the data cleaning, there are 11266 samples in dataset A and 6175

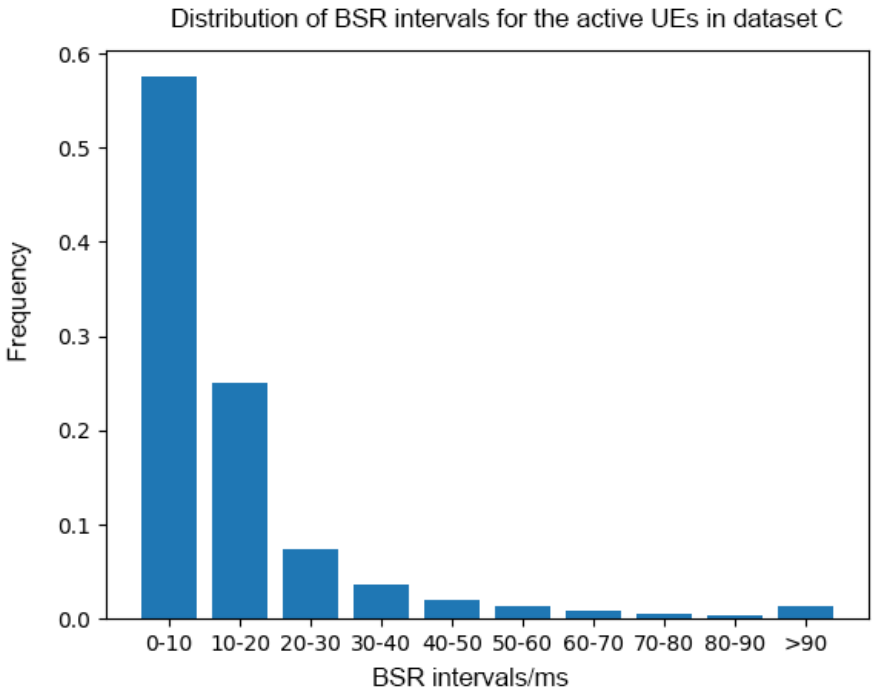


Figure 4.2 – Distribution of BSR intervals for the active UEs in dataset C. The X-axis shows the range of intervals between two adjacent BSRs, and the unit is ms.The Y-axis shows the corresponding frequency.

samples in dataset B. We randomly place the cleaned data in the sets and train the datasets separately. Finally, to improve the model's generalization ability, we combine datasets A and B and get the new dataset C. The distribution of BSR intervals for the active UEs in dataset C is shown in Fig 4.2.

4.2.1 K-Fold Cross-Validation

One of the most common approaches to evaluate the prediction performance is to separate the dataset into the training set and test set. The samples in the training set are randomly selected from the original dataset without putting them back to the initial dataset. The rest of the data in the original set serves as the test set, which is not used during the training. First, the model is trained by the training set. After training, the model is used to predict the output given the input features from the test set. By comparing the output from the model and the real data, the performance can be quantized and evaluated.

During the training process, the problem of overfitting frequently appears, which is, the model can match the training data well, but cannot predict the data outside the training set well. One of the solutions to this problem is K-Fold Cross-Validation, which works as follows. First, it is needed to arrange the samples in random order. Then, the original data is divided into K groups. Every subset serves as the test set respectively, and the remaining subset data of $K-1$ groups are used as the training set. The cross-validation error is the average error of the K models.

Mean Absolute Error(MAE) and Root Mean Squared Error(RMSE) are used to evaluate errors of the models [25], where MAE is the mean of the absolute error and RMSE is square root of the mean square difference between the predicted value and the actual observed value. MAE and RMSE can be expressed as

$$MAE = \frac{\sum_{i=1}^{n} |y_i - \hat{y}_i|}{n}, \tag{4.5}$$

$$RMSE = \sqrt{\frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{n}}, \tag{4.6}$$

where y_i is the actual value and $\hat{y}_i$ is the predicted value.

4.2.2 R^2 scores

The MAE and RMSE are the absolute values of the errors. For the same test dataset, the model with lower errors has better prediction performance. However, for different datasets, it is hard to compare the performance based on the absolute values. Therefore, some other evaluation criteria are needed. We can use the coefficient of determination, noted as R^2 scores, to reflect the degree of how the input x explains the change in output y, where

$$R^2 = 1 - \frac{RSS}{TSS}. \tag{4.7}$$

For the output y, we denote by $\bar{y}_i$ and $\hat{y}_i$ its average and prediction, respectively. RSS is the residual sum of squares, where $RSS = \sum (y_i - \hat{y}_i)^2$. TSS is the total sum of squares, where $TSS = \sum (y_i - \overline{y_i})^2$.

The model has better performance if the R^2 is closer to 1. For example, $R^2 = 0.5$ means 50 % of the target variable is predicted by the input variable.

Table 4.4 – Parameters in RF

Configuration	Number of total features	Max features in a tree
BSR	5	2
BSR and DL	10	3
ALL FEATURES	12	3

4.3 Random Forest

In what follows we present the results obtained for the RF for the values of the parameters illustrated in Table 4.4. Since RF is based on bagging, one can expect that the number of subsets/trees influences the performance. In particular, one can expect that the small number of trees leads to underfitting problems, while a too large number of trees may not be able to improve the performance significantly.

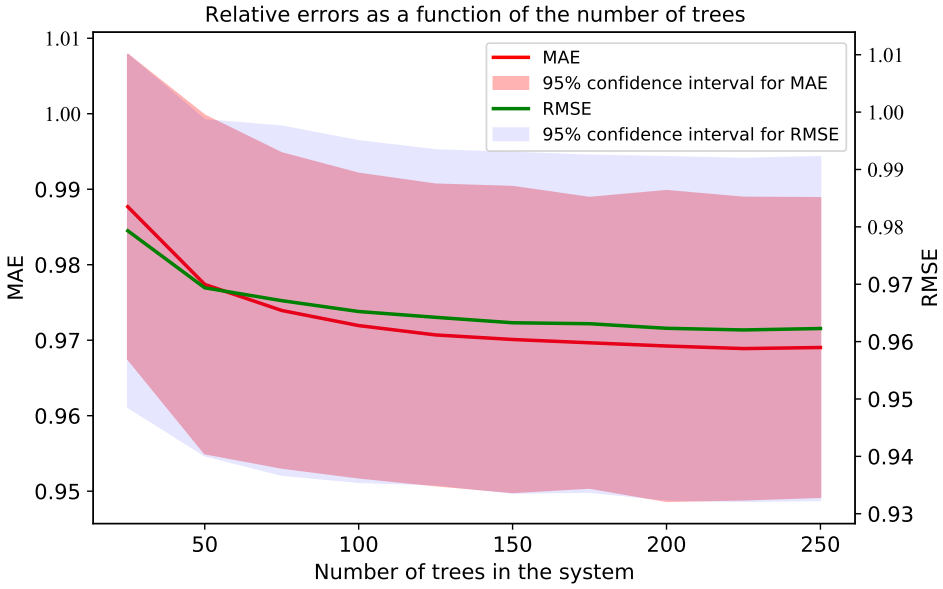


Figure 4.3 – Relative error performance as a function of the number of trees for dataset C. The X-axis shows the number of trees in the RF system.The Y-axis shows the relative error, which is the ratio of current error and maximum error.

In Figure 4.3 we show the MAE and RMSE as a function of the number

of trees. Due to the randomness of RF, the results are indeterminate. Consequently, we repeat the training process for 15 times and get the mean values and 95% confidence intervals for MSE and RMSE. As expected, the performance stops improving after a significantly large number of trees (e.g., approx 150 trees). Hence, we decide to set the number of trees to 150.

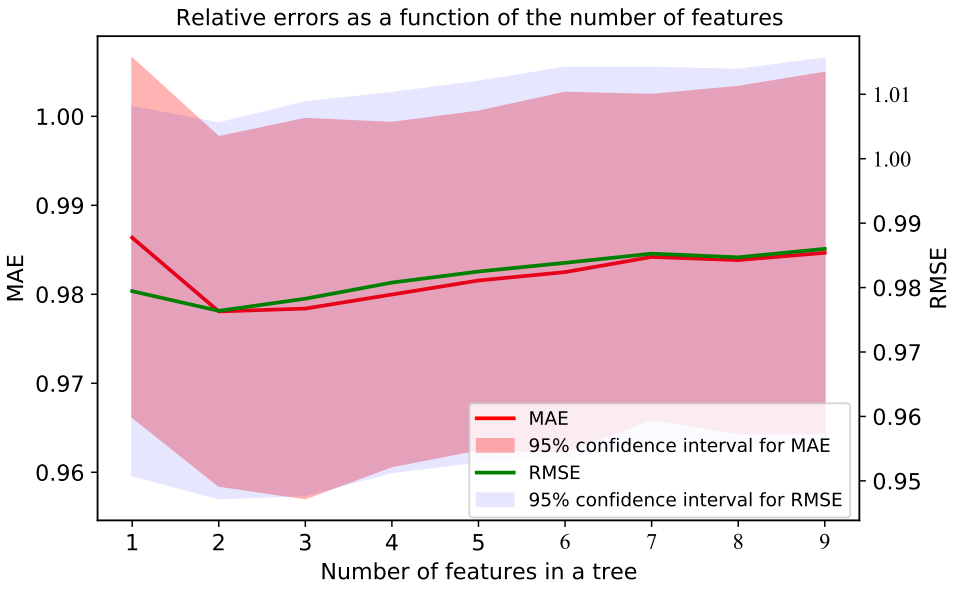
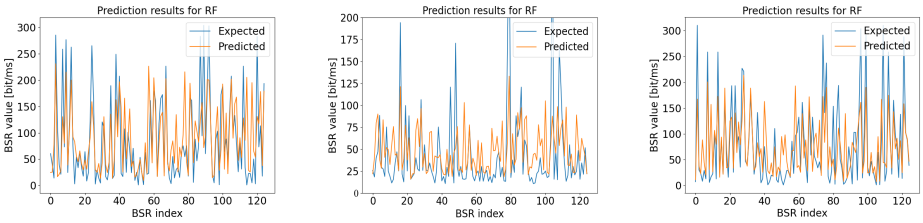


Figure 4.4 – Relative error performance as a function of the number of features in a tree for dataset C. The X-axis shows the number of features in one tree.The Y-axis shows the relative error, which is the ratio of current error and maximum error.

Another parameter of RF is the maximum number of features for each tree. The impact of the maximum number of features on the MAE and RMSE is shown in Fig 4.4 when the number of trees in this system set to 150. Due to the randomness of RF, the results are indeterminate. Consequently, we repeat the training process for 15 times and get the mean values and 95% confidence intervals for MSE and RMSE. We can observe that the performance improves, and gets degraded after a small number of features (e.g., approx 3 features). The authors in [24] find this property and suggest $log_2 T$ features can be used in each tree, where T is the number of input features. Consequently, we decide to set the number of trees to the first integer less than $log_2 T$, which is depicted in Table 4.4.

In the following, we compare the performance with different inputs. A



(a) Results for dataset A (b) Results for dataset B (c) Results for dataset C

Figure 4.5 – Prediction results of RF for respective datasets. The X-axis shows the index number of test data. The Y-axis shows BSR rates, expressed in bit/ms.

5-fold cross validation is performed and the original dataset is divided into 5 subsets. $Error_i$ is the error in the case when the i-th subset serves as test data and the remaining subsets serve as the training set.

Table 4.5 – Error of RF for dataset A

Configuration	Type of Error	Error1	Error2	Error3	Error4	Error5	Average
BSR	MAE	45.27	45.47	45.28	45.44	43.81	45.05
BSR and DL	MAE	**43.72**	**44.40**	**43.68**	**44.18**	**42.88**	**43.77**
BSR	RMSE	63.80	64.33	64.38	63.62	63.34	63.90
BSR and DL	RMSE	**61.73**	**62.90**	**62.46**	**61.82**	**61.93**	**62.17**

Table 4.6 – Error of RF for dataset B

Configuration	Type of Error	Error1	Error2	Error3	Error4	Error5	Average
BSR	MAE	30.87	31.16	31.61	32.34	31.18	31.43
BSR and DL	MAE	28.50	29.70	29.86	30.90	29.57	29.71
ALL FEATURES	MAE	**28.40**	**29.69**	**29.66**	**30.81**	**29.19**	**29.55**
BSR	RMSE	48.67	49.82	49.64	49.97	48.55	49.33
BSR and DL	RMSE	46.15	48.67	47.47	48.73	46.07	47.42
ALL FEATURES	RMSE	**45.83**	**48.31**	**47.38**	**48.04**	**45.70**	**47.05**

Table 4.7 – Error of RF for dataset C

Configuration	Type of Error	Error1	Error2	Error3	Error4	Error5	Average
BSR	MAE	40.75	39.97	40.70	40.90	40.56	40.58
BSR and DL	MAE	**39.22**	**38.53**	**39.64**	**39.47**	**39.27**	**39.23**
BSR	RMSE	59.88	57.79	60.15	59.93	59.41	59.43
BSR and DL	RMSE	**58.26**	**56.40**	**58.63**	**58.19**	**57.98**	**57.89**

Table 4.5, 4.6 and 4.7 show the errors of RF for datasets A, B and C, respectively, for the set of parameters summarized in Table 4.4. We can

observe that for all of the cases, both the MAE and RMSE get reduced if the input involves more parameters. This indicates that for RF, the downlink data, downlink quality indicator, and rank indicator can improve the prediction performance.

Fig 4.5 shows the value of BSR as a function of the index of BSR for the expected and predicted results of RF for datasets A, B and C, respectively. We can observe that for dataset A, the predicted results of RF match best with the expected value. When the BSR rates are low, the predicted values are close to the real values. When the BSR rates are local maxima, the predicted results are local maxima too. For dataset B, this RF model shows the worst performance. When the BSR rates are low, the predicted values tend to be a little bigger than the actual values. For some samples (e.g., approx the $52nd$ sample), the predicted result is the local maximum while the expected value is much smaller. Dataset A and B are collected from different companies, and the users in different companies may have different demands on network data. For example, the users in dataset A prefer to upload more data than the users in dataset B and it is easier to predict BSR data for communication scenarios in dataset A.

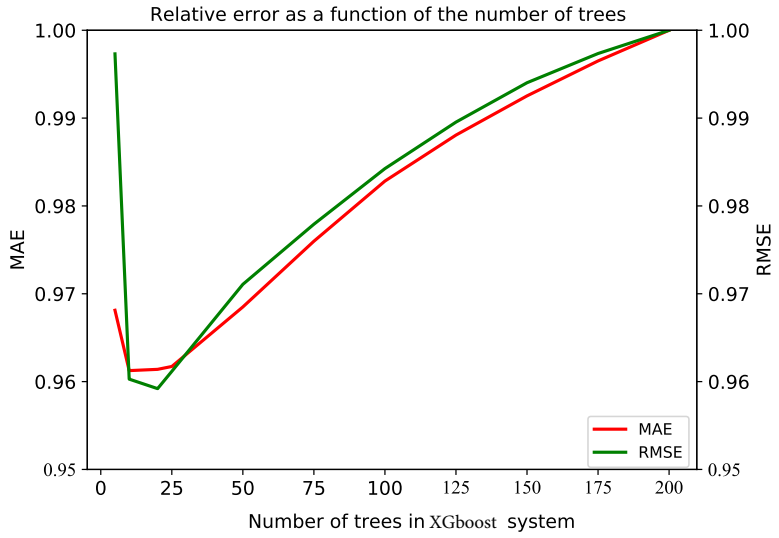


Figure 4.6 – Performance as a function of the number of trees in XGboost for dataset C

4.4 XGboost

In the following, we present the results obtained for XGboost with the number of trees in this model set to 25. Different from RF, which is based on bagging, XGboost is based on boosting. The increasing number of trees in this system cannot constantly improve the performance of prediction. In particular, a large number of trees may lead to overfitting problems. In Fig 4.6 we show the relative MAE and RMSE as a function of the number of trees in a XGboost system. As expected, the performance starts degrading after a significantly large number of trees (e.g., approx 25 trees). Consequently, we decide to set the number of trees in XGboost system to 25.

Table 4.8 – Errors of XGboost for dataset A

Configuration	Type of Error	Error1	Error2	Error3	Error4	Error5	Average
BSR	MAE	44.34	44.59	44.54	44.98	43.30	44.35
BSR and DL	MAE	**42.96**	**43.80**	**43.10**	**43.85**	**42.55**	**43.25**
BSR	RMSE	63.20	64.15	64.36	63.27	63.40	63.68
BSR and DL	RMSE	**62.03**	**63.40**	**62.66**	**62.00**	**62.56**	**62.53**

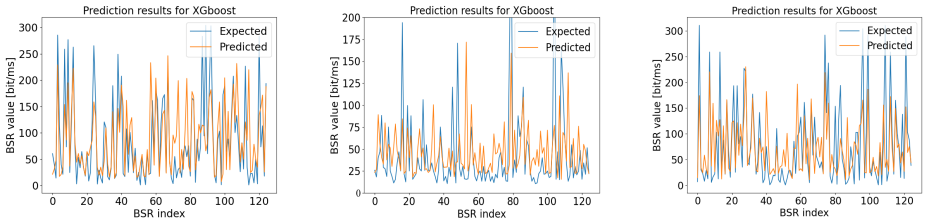
Table 4.9 – Errors of XGboost for dataset B

Configuration	Type of Error	Error1	Error2	Error3	Error4	Error5	Average
BSR	MAE	29.48	30.81	31.24	32.37	30.46	30.87
BSR and DL	MAE	29.06	**29.28**	30.51	31.72	29.30	29.97
ALL FEATURES	MAE	**28.93**	29.82	**30.14**	**30.87**	**29.17**	**29.79**
BSR	RMSE	49.09	51.05	51.07	52.33	48.94	50.50
BSR and DL	RMSE	48.09	**49.91**	50.30	51.13	47.53	49.39
ALL FEATURES	RMSE	**48.59**	50.84	**49.12**	**50.39**	**47.25**	**49.24**

Table 4.10 – Errors of XGboost for dataset C

Configuration	Type of Error	Error1	Error2	Error3	Error4	Error5	Average
BSR	MAE	40.12	38.87	40.37	40.51	39.61	39.90
BSR and DL	MAE	**38.65**	**37.33**	**39.27**	**39.24**	**38.99**	**38.70**
BSR	RMSE	59.87	57.11	60.24	60.31	58.91	59.29
BSR and DL	RMSE	**58.40**	**56.02**	**59.25**	**58.85**	**58.54**	**58.21**

In what follows a 5-fold cross validation is performed. Table 4.8, 4.9 and 4.10 show the MAE and RMSE of XGboost for dataset A, B and C, respectively, with the number of trees in this system set to 25. We can observe that for most of the cases, both the MAE and RMSE get reduced if the input involves more parameters. It indicates that for XGboost, the



(a) Results for dataset A (b) Results for dataset B (c) Results for dataset C

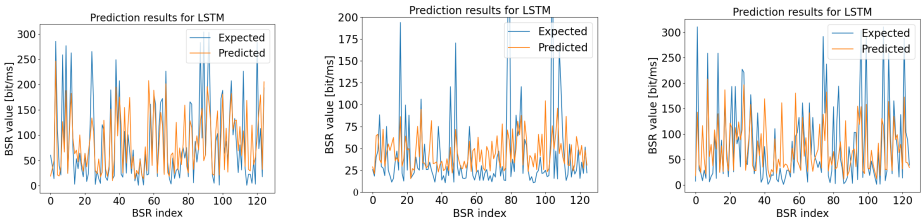
Figure 4.7 – Prediction results of XGboost for respective datasets. The X-axis shows the index number of test data. The Y-axis shows BSR rates, expressed in bit/ms.

downlink data, downlink quality indicator, and rank indicator can improve the prediction performance. In addition, there is only a slight difference between the performance of XGboost and RF except that for dataset B, where the RMSE of XGBoost is considerably higher than that of RF. This is because the RF is based on bagging and XGboost is based on boosting.

Fig 4.7 shows the value of BSR as a function of the index of BSR for the expected and predicted results of XGboost for datasets A, B and C, respectively. We can observe that similar to RF, XGboost has best performance for dataset A and worst performance for dataset B. The performance of XGboost for dataset C is between the performance for dataset A and B. This is because it is easier to predict the behaviors for UE in dataset A than B, and dataset C contains all samples in dataset A and B.

4.5 LSTM

In what follows, we present the results obtained for LSTM with the number of hidden units set to 150. For LSTM, one can expect that the training epochs influence the performance. Typically, the performance is better with more training computation and time. However, when the input has more features, it requires more computation and takes more time to converge for the fixed training epochs. In order to provide a fair comparison, we show the results for different inputs with the same training loss. The authors in [26] point out that the Mean Squared Error (MSE) loss function can serve as a good general-purpose loss function in option valuation applications. Consequently,



(a) Results for dataset A (b) Results for dataset B (c) Results for dataset C

Figure 4.8 – Prediction results of LSTM for respective datasets. The X-axis shows the index number of test data. The Y-axis shows BSR rates, expressed in bit/ms.

we choose MSE function as our loss function, which can be expresses as

$$L(y, \hat{y}) = \frac{1}{N} \sum_{i=0}^{N} (y_i - \hat{y}_i)^2, \tag{4.8}$$

where N is the number of samples, y_i and $\hat{y}_i$ are the expected and predicted values for the i-th sample, respectively. As mentioned in Chapter 3.1.2, the LSTM unit has to transfer the hidden state h to the next unit. Therefore, the dimension of hidden state h is significant to the performance of the final results. According to [27], the hidden state stores essential information of all the input sequence data. Therefore, it is common to consider the hidden state as the memory of the LSTM. It is intuitive to expect an improvement in performance when we increase the size of the hidden state up to some extent. However,the results in [27] show a too large number of hidden units may not be able to improve the performance significantly. One of the possible reasons is that the features of input are fixed. As the dimension of the hidden units is much higher than the dimension of input, the increasing of hidden units will have little impact on the performance. We run our model with different hidden state size of 50, 100, 150, 200 and 250. As expected, the performance stop improving after a significantly large number of units (e.g., approx 150 units). Consequently, we decide to set the number of hidden units to 150. A training data normalization is implemented to boost the performance of this LSTM model. According to [28], the normalization can help stabilize the gradient magnitude in training LSTM. Therefore, normalized LSTMs can achieve significantly better results than their unnormalized counterparts.

Table 4.11, 4.12 and 4.13 show the MAE and RMSE of LSTM for dataset A, B and C, respectively. We can observe that except in the case of dataset B, the MAE and RMSE decrease with the number of input features. For dataset

Table 4.11 – Errors of LSTM for dataset A

Configuration	Type of Error	Error1	Error2	Error3	Error4	Error5	Average
BSR	MAE	44.70	45.18	44.38	45.32	43.23	44.56
BSR and DL	MAE	**44.58**	**45.11**	**44.16**	**45.29**	**43.05**	**44.44**
BSR	RMSE	63.23	64.23	63.91	63.83	63.14	63.67
BSR and DL	RMSE	**63.01**	**64.33**	**63.42**	**63.76**	**63.12**	**63.53**

Table 4.12 – Errors of LSTM for dataset B

Configuration	Type of Error	Error1	Error2	Error3	Error4	Error5	Average
BSR	MAE	**28.98**	29.41	29.66	**30.81**	29.46	29.67
BSR and DL	MAE	29.08	29.46	**29.35**	30.92	29.52	29.66
ALL FEATURES	MAE	29.05	**29.36**	29.65	30.85	**29.16**	**29.61**
BSR	RMSE	**47.31**	**48.15**	47.88	**48.76**	**46.22**	**47.66**
BSR and DL	RMSE	48.16	48.44	**47.32**	49.30	46.46	47.94
ALL FEATURES	RMSE	47.96	48.63	47.69	49.30	46.26	47.97

Table 4.13 – Errors of LSTM for dataset C

Configuration	Type of Error	Error1	Error2	Error3	Error4	Error5	Average
BSR	MAE	42.48	39.76	41.97	41.89	41.39	41.50
BSR and DL	MAE	**41.93**	**39.66**	**41.86**	**41.29**	**40.90**	**41.13**
BSR	RMSE	61.54	57.96	61.15	60.35	59.88	60.18
BSR and DL	RMSE	**61.47**	**57.74**	**60.65**	**60.48**	**59.93**	**60.05**

B, the performance degrades with more input features. One of the possible reasons according to [29] is that the less important features get higher weights in the neural network. The authors of [29] suggest to implement an attention model to efficiently extract the features of the data and use them as input to LSTM deep learning model.

Fig 4.8 shows the part of the prediction results of LSTM for datasets A, B and C, respectively. We can observe that similar to RF and XGboost, LSTM has best performance for dataset A and worst performance for dataset B. For dataset A, the predicted results match well with the expected value. For dataset B, compared with RF and XGboost models, the LSTM is more conservative. In most of the cases, the predicted values are larger than the expected value when the expected value is small, and the predicted values are smaller than the expected value when the expected value is large. One of the possible explanations is that the predicted values may be around the average value of the samples if it is hard to make accurate prediction, in order to minimize the loss function. The performance of LSTM for dataset C is between the performance for dataset A and B since dataset C contains all samples in dataset A and B.

Chapter 5

Discussion

Table 5.1 shows the best performances of the three algorithms with their corresponding training time. The unit of training time is expressed in seconds and the CPU we use is Intel Core i7-8650U.

Table 5.1 – Performance of different algorithms

Dataset	Algorithms	MAE	RMSE	Training Time
A	RF	43.77	**62.17**	16.74
A	XGboost	**43.25**	62.53	**0.63**
A	LSTM	44.44	63.53	17522
B	RF	**29.55**	**47.05**	11.54
B	XGboost	29.79	49.24	**0.43**
B	LSTM	29.61	47.97	5426
C	RF	39.23	**57.89**	22.64
C	XGboost	**38.70**	58.21	**1.03**
C	LSTM	41.13	60.05	6490

First, from previous results we can observe that in most of the cases, models have better performance if the inputs include more features. This is because the downlink data is co-related to the uplink data, and downlink quality indicator and rank indicator are related to the priority of the packets, which impacts the trigger of BSRs.

As we mentioned before, typically bagging can reduce the variance and boosting can decrease the bias. The RF model, which is based on bagging, has the lowest RMSE among the algorithms presented in Table 5.1. For RF, We observe that a small number of trees in the case of an RF model may lead to underfitting problems, which can be addressed by adding more trees to the

model. However, in the case of an RF model with a large number of trees, the increase of trees cannot dramatically enhance the prediction results.

The results in Table 5.1 also suggest that in most of the cases, the XGboost model, which is based on boosting, has the lowest MAE. Similar to RF, the XGboost model faces the underfitting problem if the number of regression trees is small. Different from RF, the increase of trees may have a negative impact on the final results, which we call overfitting problem. This is because the weights of the training samples are influenced by the training results from previous trees.

The XGboost is known for its accuracy and high efficiency. In our 5-fold cross validation process, it takes the least time to train the XGboost models. As for the LSTM, the model performance is closely related to the training time. In our 5-fold cross validation process, the time LSTM consumes is hundred times of time XGboost and RF consume. In addition, with more computation and time consumed, the error of LSTM is much higher than XGboost and RF models. In practical terms, models have to be trained periodically by the newly collected data to adapt to the new environment. Consequently, the training time is one of the important factors.

Table 5.2 shows the R^2 scores for dataset C. The R^2 values for our models are around 0.5. According to [30], we assume the relationship is moderate when the R^2 value is around 0.5. And the relationship is strong when the $R^2 > 0.7$.

Table 5.2 – R^2 scores for dataset C

Algorithms	R^2 **score**
RF	**0.5135**
XGboost	0.5077
LSTM	0.4710

In addition, the data in dataset A is quite different from dataset B. This is because they are collected from different companies and their users have different demands of service. For dataset C, which is combined from dataset A and B, our models still show good prediction performance, which indicates that our models can adapt to different environments.

Chapter 6

Conclusions and Future Work

In this chapter, we summarize the work we have done and discuss possible future work directions.

6.1 Conclusions

The focus in this thesis is on predicting the data arrival rate in short time intervals. The purpose of this work is to allow base stations to allocate more resources to the UEs and by doing so to shorten their latencies. To the best of our knowledge, we are the first to investigate the uplink data arrival rate in such short time intervals without making assumptions about the data distributions.

Compared with other network data forecasting, the prediction in this thesis is more challenging due to its short time intervals. When the intervals are large, many macroscopical features can be leveraged to predictions. For example, the data collected at the same time on different days can be used to explore the periodicity, and the data collected during the same workday in different weeks can be used to explore the periodicity trend. On the contrary, when the time intervals are short, we can assume the macroscopical features keep unchanged. Therefore, they have little impact to the predictions.

Finally, we decide to choose a time window and predict the data at the subsequent time intervals based on the records in the time window. Although ARIMA is widely used in time series forecasting, we do not implement it in our thesis. This is because, there are many UEs, and the length of their data may vary significantly. Therefore, it is hard to find the coefficients that work well for all UEs. After some necessary data preprocessings, we implement state-of-the-art machine learning algorithms such as Random Forest, XGboost, and LSTM. The simulation results show that the implemented algorithms can

achieve good performance.

For the questions propoesed Charpter 1, the results in our thesis show that:

- Some state-of-the-art machine learning algorithms such as RF, XGboost and LSTM have the potential to make accurate traffic predictions even when the time intervals are small.
- Except the previous BSR information, there are some other features can be leveraged as the input of our model. Our prediction results show that the downlink data, whose distribution is correlative to the distribution of uplink data, can improve the performance. In addition, the rank indicator and downlink quality indicator can reflect the priority of the data, which also boost the prediction performance.
- In this thesis, we find the relationship between two BSRs is weak if their interval is long. So we divide the time series from the same UE into many sub-series and form a labeled dataset based on the time series in order to train our supervised learning model.

6.2 Limitations

Although we can make BSR rate predictions via RF, XGboost and LSTM, our thesis has several limitations due to different reasons. One of the limitations is concerning the datasets. There are are only few available datasets related to the downlink, uplink, and BSR data since they are hard to collect. Hence, our work is based on the datasets provided by Ericsson, collected from the real network. Though there are many data samples in the datasets, after data cleaning, the total data samples are only about 20000 because most of the UEs do not send BSR frequently. Furthermore, many features can influence the arrival data volume, but only part of them are recorded in the dataset. For example, for uplink data, only the value of BSR is recorded. Consequently, the length of time intervals between the moments when a UE sends BSR and transmits the uplink data is unknown. Therefore, we use the BSR rate to indicate the data arrival rate. The advantage of this method is that if one can predict the data arrival rate successfully, we can get the arrival data volume since the time intervals between the times when BSR is sent and when the uplink data is transmitted are easy to collect. However, the drawback of this method is that the average data arrival rate may not make accurate predictions of the real traffic data in the cases when time intervals are tremendously short (e.g., in the order of milliseconds), and the data occurs in bursts. For example, when

the interval is 5ms and the rate is 200 bit/ms, the predicted data volume is 1kb. However, in practice if there are no packets arriving during this interval, the volume should be 0. If there are packets arriving during this interval, the data volume may be much larger than 1kb.

Finally, the start of the thesis was postponed due to the pandemic, and hence the time on the problem was shorter than initially planned. Therefore, only the simulation experiments for BSR rate prediction are done. The simulation experiments for End-to-End latency after implementing BSR prediction are missing. Therefore, we didn't provide the latency of data delivery as the ultimate performance metric of this thesis.

6.3 Future work

In the future, some tasks can be done to improve the uplink scheduling. For example, one of the observations based on the work in this thesis is that the LSTM model does not perform well in terms of MAE and RMSE compared to RF and XGboost models. Furthermore, no significant improvement was observed after adding more layers and units to the LSTM. In the future, we can combine LSTM attention or transformer layer to the predictor to enhance the performance.

Another observation based on the coefficient of determination performance is that the relationship between the target variable and the input variable is not strong. One of the reasons is that the data samples are few. In the future, we can collect data with features in the data link, network and transport layers. In addition, we may be able to address the problem of few samples via few shot type of learning. The simulator also can provide us many valuable data.

In this thesis, the simulation experiments for End-to-End latency after implementing BSR prediction are missing. In the future, we can implement our prediction models to the simulator or real traffic network to check the latency performance.

References

[1] Ericsson, "Mobile data traffic outlook," Website, 2021, https://www.ericsson.com/en/mobility-report/dataforecasts/mobile-traffic-forecast.

[2] E. U. T. R. Access, "Medium access control (mac) protocol specification (release 11), 3gpp ts 36.321, jun. 2013," *V11*, vol. 3.

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, pp. 1735–80, 12 1997.

[4] H. yi Lee, "Gradient descent as lstm," Website, 2020, https://speech.ee.ntu.edu.tw/~tlkagk/courses/ML_2019/Lecture/Meta2%20(v4).pdf.

[5] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *nature*, vol. 323, no. 6088, pp. 533–536, 1986.

[6] L. I. Kuncheva and C. J. Whitaker, "Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy," *Machine learning*, vol. 51, no. 2, pp. 181–207, 2003.

[7] P. S. A. Krogh *et al.*, "Learning with ensembles: How over-fitting can be useful," in *Proceedings of the 1995 Conference*, vol. 8, 1996, p. 190.

[8] L. Breiman, "Bagging predictors," *Machine learning*, vol. 24, no. 2, pp. 123–140, 1996.

[9] M. Kearns and L. Valiant, "Cryptographic limitations on learning boolean formulae and finite automata," *Journal of the ACM (JACM)*, vol. 41, no. 1, pp. 67–95, 1994.

[10] L. Breiman, J. Friedman, C. J. Stone, and R. Olshen, *Classification and Regression Trees*. Chapman and Hall/CRC, 1984.

[11] L. Lin, F. Wang, X. Xie, and S. Zhong, "Random forests-based extreme learning machine ensemble for multi-regime time series prediction," *Expert Systems with Applications*, vol. 83, pp. 164–176, 2017.

[12] T. Chen and C. Guestrin, "Xgboost," *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, Aug 2016.

[13] N. Patriciello, S. Lagen, L. Giupponi, and B. Bojovic, "The impact of nr scheduling timings on end-to-end delay for uplink traffic," in *2019 IEEE Global Communications Conference (GLOBECOM)*, 2019, pp. 1–6.

[14] C. Fei, B. Zhao, C. Wu, W. Yu, B. Han, and J. Su, "Towards efficient radio resource scheduling in lte-based satellite mobile communication system," in *2015 IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, 2015, pp. 167–172.

[15] E. W. Wallentinsson, "Multiple time series forecasting of cellular network traffic," Master's thesis, Linköping University, The Division of Statistics and Machine Learning, 2019.

[16] B. Zhou, D. He, Z. Sun, and W. H. Ng, "Network traffic modeling and prediction with arima/garch," in *Proc. of HET-NETs Conference*, 2005, pp. 1–10.

[17] O. Triebe, N. Laptev, and R. Rajagopal, "Ar-net: A simple auto-regressive neural network for time-series," *arXiv preprint arXiv:1911.12436*, 2019.

[18] A. Azari, P. Papapetrou, S. Denic, and G. Peters, "Cellular traffic prediction and classification: A comparative evaluation of lstm and arima," in *International Conference on Discovery Science*. Springer, 2019, pp. 129–144.

[19] J. Lee, S. Lee, J. Lee, S. D. Sathyanarayana, H. Lim, J. Lee, X. Zhu, S. Ramakrishnan, D. Grunwald, K. Lee, and S. Ha, "Perceive: deep learning-based cellular uplink prediction using real-time scheduling patterns," *Proceedings of the 18th International Conference on Mobile Systems, Applications, and Services*, 2020.

[20] H. D. Trinh, L. Giupponi, and P. Dini, "Mobile traffic prediction from raw data using lstm networks," in *2018 IEEE 29th Annual International*

Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC), 2018, pp. 1827–1832.

[21] Q. He, A. Moayyedi, G. Dán, G. P. Koudouridis, and P. Tengkvist, "A meta-learning scheme for adaptive short-term network traffic prediction," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 10, pp. 2271–2283, 2020.

[22] M. Kumar and M. Thenmozhi, "Forecasting stock index movement: A comparison of support vector machines and random forest," in *Indian institute of capital markets 9th capital markets conference paper*, 2006.

[23] X. Dong, T. Lei, S. Jin, and Z. Hou, "Short-term traffic flow prediction based on xgboost," in *2018 IEEE 7th Data Driven Control and Learning Systems Conference (DDCLS)*, 2018, pp. 854–859.

[24] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.

[25] T. Hastie, R. Tibshirani, R. Tibshirani, and J. H. Friedman, "The elements of statistical learning: Springer series in statistics," 2001.

[26] P. Christoffersen and K. Jacobs, "The importance of the loss function in option valuation," *Journal of Financial Economics*, vol. 72, no. 2, pp. 291–318, 2004.

[27] Y. Shi, B. Fernando, and R. Hartley, "Action anticipation with rbf kernelized feature mapping rnn," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 301–317.

[28] L. Hou, J. Zhu, J. T.-Y. Kwok, F. Gao, T. Qin, and T.-y. Liu, "Normalization helps training of quantized lstm," 2019.

[29] X. Zhang, X. Liang, A. Zhiyuli, S. Zhang, R. Xu, and B. Wu, "At-lstm: An attention-based lstm model for financial time series prediction," in *IOP Conference Series: Materials Science and Engineering*, vol. 569, no. 5. IOP Publishing, 2019, p. 052037.

[30] D. S. Moore and S. Kirkland, *The basic practice of statistics*. WH Freeman New York, 2007, vol. 2.

For DIVA


```
{
"Author1": {
            "Last name": "Zhang",
            "First name": "Qi ",
            "E-mail": "qizhan@kth.se",
            "organisation": {"L1": "School of Electrical Engineering and Computer Science ",
                                }
          },
"Degree": {"Educational program": "Master's Programme, Information and Network Engineering, 120 credits"},
"Title": {
            "Main title": "Master thesis",
            "Subtitle": "Predicting Buffer Status Report (BSR) for 6G Scheduling using Machine Learning Models",
            "Language": "eng" },
"Alternative title": {
            "Main title": "",
            "Language": "swe"
},
"Supervisor1": {
                "Last name": "Josilo",
                "First name": "Sladana",
                "Other organisation": "KTH, Department of EECS"}
              },
"Supervisor2": {
                "Last name": "Daoutis",
                "First name": "Marios",
                "Other organisation": "Ericsson AB"}
              },
"Supervisor3": {
                "Last name": "Nikou",
                "First name": "Alexandros",
                "Other organisation": "Ericsson AB"}
              },
"Examiner1": {
                "Last name": "Dán",
                "First name": "Gyorgy",
                "Local User Id": "u100004",
                "E-mail": "gyuri@kth.se",
                "organisation": {"L1": "School of Electrical Engineering and Computer Science ",
                                  "L2": "Computer Science" }
              },
"Cooperation": { "Partner_name": "Ericsson AB"},
"Other information": {
"Year": "2021", "Number of pages": "15,47"}
}
```

TRITA-EECS-EX-2021:912